Exercise 16: Default Value Fallback

Project documentation: NvM block CRC verification with compile-time default fallback

Platform: S32K1xx MCU (S32 SDK: CRC, LPUART, PINS, CLOCK drivers)

1. Problem Statement

If a block's CRC check fails and no redundant copy exists, load a compile-time default value into RAM instead of leaving it uninitialized. Notify the application via `NvM_DefaultsUsedFlag = 1`.

2. Objective

- Simulate a Non-Volatile Memory (NvM) block holding vehicle configuration data.
- Compute and store a CRC for the block, then re-verify it at ECU startup.
- If the CRC matches, load the NvM data into the active RAM configuration.
- If the CRC does not match, load the compile-time default configuration into RAM instead, so the application never runs on uninitialized data.
- Inform the application through the flag `NvM_DefaultsUsedFlag` (0 = NvM data used, 1 = defaults used).
- Report the result over UART and indicate it on the on-board LEDs.

3. Background

In AUTOSAR-style memory management, the NvM module protects each block with a CRC. When a read-back fails the integrity check (`NVM_REQ_INTEGRITY_FAILED`) and the block has no redundant copy, the standard recovery strategy is a **ROM default fallback**: the block's RAM mirror is filled from constant defaults stored in flash, and the application is notified that defaults are in use so it can react (for example by logging a diagnostic or limiting functionality).

This project demonstrates that mechanism on a microcontroller, with the NvM block simulated in RAM.

4. Hardware and Software Environment

Item	Details
MCU	NXP S32K1xx series (S32K144 EVB assumed from pin usage)
SDK	S32 SDK drivers: `CRC_DRV`, `LPUART_DRV`, `PINS_DRV`, `CLOCK_DRV`
IDE	S32 Design Studio
Serial terminal	Any terminal (PuTTY, Tera Term) on the LPUART COM port
Language	Embedded C

5. Design

5.1 Data Structure

`VehicleConfig_t` holds the vehicle parameters:

Field	Type	Meaning
`maxSpeedLimit`	`uint16_t`	Speed limit in km/h
`engineTempLimit`	`uint8_t`	Engine temperature limit in °C
`minBatteryVoltage`	`uint16_t`	Minimum battery voltage in 0.1 V units (120 = 12.0 V)
`headlightAutoMode`	`uint8_t`	0 = OFF, 1 = ON

5.2 Key Variables

Variable	Role
`Default_VehicleConfig`	`const` compile-time default values (stored in flash)

Variable	Role
`Active_VehicleConfig`	Configuration actually used by the application (RAM)
`NvM_Block`	Simulated NvM block content
`storedCRC`	CRC saved when the block was "written"
`NvM_DefaultsUsedFlag`	Notification flag to the application: 0 = NvM data valid, 1 = defaults loaded
`txBuff[200]`	UART transmit buffer

5.3 Data Values

Parameter	Default (flash)	NvM block (simulated)
Max speed limit	120 km/h	100 km/h
Engine temperature limit	105 °C	100 °C
Min battery voltage	12.0 V (120)	11.5 V (115)
Headlight auto mode	ON	OFF

The two sets are intentionally different, so the console output shows clearly which set was loaded.

5.4 Functions

`CalculateBlockCRC(dataBuffer, length)` initializes the CRC peripheral (`CRC_DRV_Init`), feeds the buffer with `CRC_DRV_WriteData` , and returns the result via `CRC_DRV_GetCrcResult` . It returns 0 if initialization fails. Because the peripheral is re-initialized on every call, each calculation starts from the configured seed value, making results repeatable.

`main()` performs the whole sequence described in section 6.

5.5 LED Indication

Result	PTD16	PTD15	Visible effect (S32K144EVB, active-low RGB LED)
CRC valid	0	1	Green LED ON, red LED OFF
CRC failed	1	0	Red LED ON, green LED OFF

The LED pin mapping assumes the S32K144EVB, where PTD15 is the red and PTD16 the green channel of the RGB LED. Adjust if your board differs.

6. Working Principle

1. Initialize clock, pins and LPUART.
2. Calculate the CRC of `NvM_Block` and keep it in `storedCRC` (simulates the CRC saved when the block was last written).
3. Optionally corrupt the block to simulate memory corruption (the commented line `NvM_Block.maxSpeedLimit ^= 0xFFFF`).
4. Send the startup banner over UART and wait until transmission completes.
5. Recalculate the CRC of `NvM_Block` (simulates the read-back check at ECU startup).
6. Compare the calculated CRC with `storedCRC`.
   - **Equal:** copy `NvM_Block` to `Active_VehicleConfig`, set `NvM_DefaultsUsedFlag = 0`, turn the green LED on, print `NVM_REQ_OK` with the configuration.
   - **Different:** copy `Default_VehicleConfig` to `Active_VehicleConfig`, set `NvM_DefaultsUsedFlag = 1`, turn the red LED on, print `NVM_REQ_INTEGRITY_FAILED` with the default configuration.
7. Transmit the result message and wait for completion.
8. Enter an infinite loop to keep the program alive.

The flowchart for this sequence is in the Program Flowchart section at the end.

7. Test Cases

#	Setup	Expected CRC result	Flag	LED	UART output
1	Normal run (corruption line commented)	Match	`NvM_DefaultsUsedFlag = 0`	Green	`NvM Block Valid: NVM_REQ_OK`, Max Speed 100 km/h, Temp 100 C, Battery 11.5 V, Headlights OFF, Defaults Used: 0
2	Uncomment `NvM_Block.maxSpeedLimit ^= 0xFFFFU;`	Mismatch	`NvM_DefaultsUsedFlag = 1`	Red	`CRC Failed: NVM_REQ_INTEGRITY_FAILED`, defaults loaded: Max Speed 120 km/h, Temp 105 C, Battery 12.0 V, Headlights ON, Defaults Used: 1, `LED ON`

Expected console output

Test case 1 (valid block):

```
--- ECU Startup: NvM CRC Check ---
NvM Block Valid: NVM_REQ_OK
Max Speed Limit: 100 km/h
Engine Temperature Limit: 100 C
Minimum Battery Voltage: 11.5 V
Automatic Headlights: OFF
Defaults Used: 0
```

Test case 2 (corrupted block):

```
--- ECU Startup: NvM CRC Check ---
CRC Failed: NVM_REQ_INTEGRITY_FAILED
Default configuration loaded into RAM
Max Speed Limit: 120 km/h
Engine Temperature Limit: 105 C
Minimum Battery Voltage: 12.0 V
Automatic Headlights: ON
Defaults Used: 1
LED ON
```

8. Requirement Traceability

Exercise requirement	Implementation
Detect CRC failure of a block	CRC recalculated and compared with `storedCRC` in `main()`
No redundant copy exists	Single `NvM_Block`, no secondary copy is read
Load compile-time default into RAM	`memcpy(&Active_VehicleConfig, &Default_VehicleConfig, ...)` with `const Default_VehicleConfig`
Do not leave RAM uninitialized	`Active_VehicleConfig` is filled in both branches
Notify application with `NvM_DefaultsUsedFlag = 1`	Flag set to 1 in the failure branch, 0 in the success branch

9. Observations and Possible Improvements

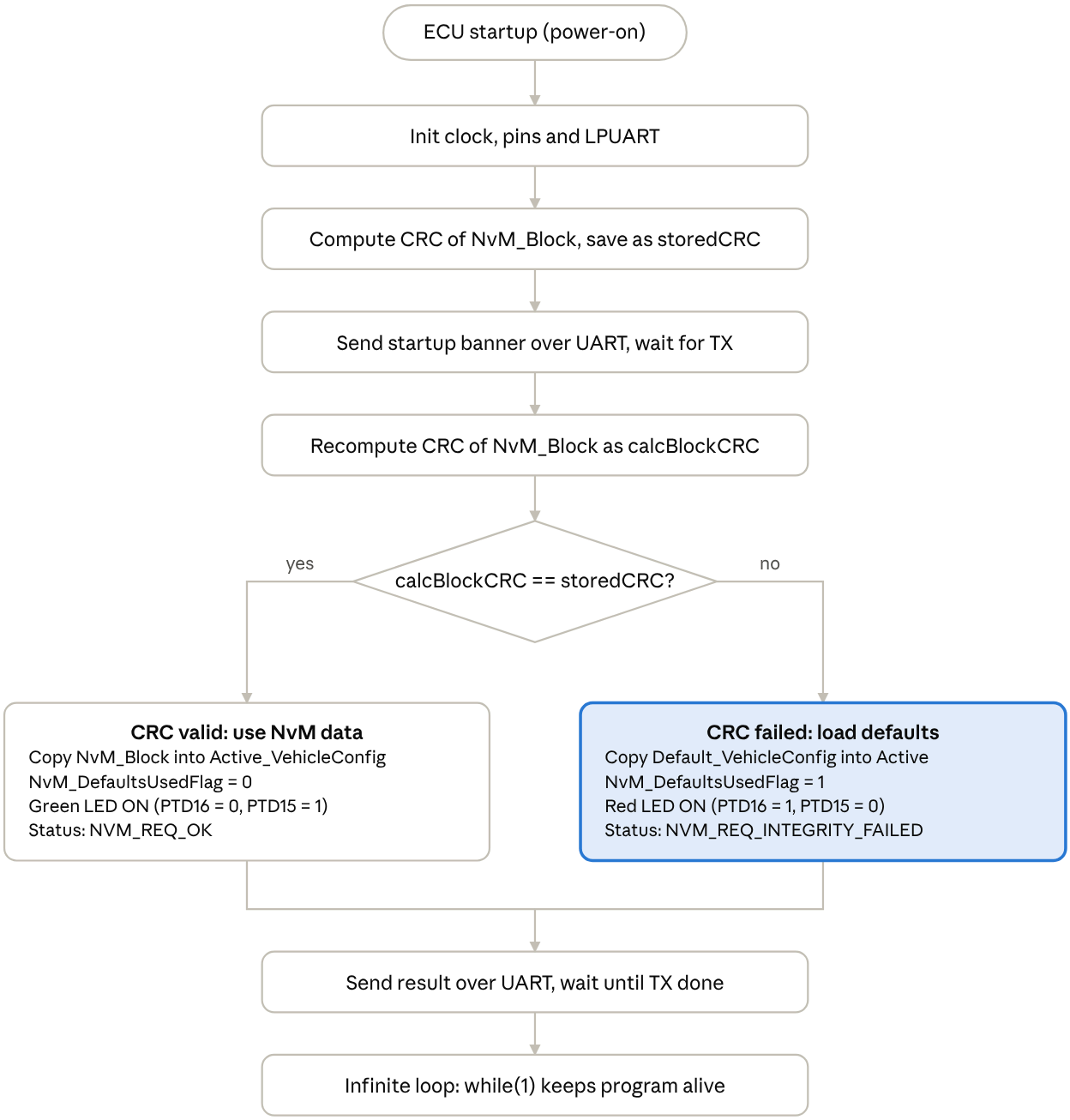
- `NVM_REQ_OK` and `NVM_REQ_INTEGRITY_FAILED` are defined but only appear in the printed strings. Using them in a return/status variable (for example `NvM_RequestResult`) would make the code closer to AUTOSAR NvM behaviour.
- The CRC covers the whole struct including compiler padding bytes (the struct is 8 bytes on this 32-bit target). It works here because the block is zero-initialized as a global, but for real NvM data a `packed` struct or a field-by-field CRC is safer.
- `CalculateBlockCRC` returns 0 on init failure, and 0 can also be a valid CRC. A separate status return would remove the ambiguity.
- `sprintf` into a fixed 200-byte buffer is safe for the current messages, but `snprintf` would guard against future edits.
- The LED pin numbers (15, 16) are used as literals for pin 16; defining a `LED_GREEN_PIN` macro would be cleaner.
- A real implementation would read the block and its CRC from flash/EEPROM instead of RAM, and write the defaults back to NvM after recovery.

10. Conclusion

The program demonstrates the default value fallback strategy: the NvM block integrity is verified with a hardware CRC at startup, and when the check fails, constant defaults are loaded into RAM so the application always starts from a defined configuration. `NvM_DefaultsUsedFlag` tells the application which source was used, while the UART log and LED provide visible confirmation for testing and demonstration.

Program Flowchart

A failed CRC check loads defaults and sets NvM_DefaultsUsedFlag = 1



program flow · 5 setup steps, 1 decision, 2 branches

Both branches fill `Active_VehicleConfig`, so it is never left uninitialized; only the failing branch (highlighted) loads the compile-time defaults and raises `NvM_DefaultsUsedFlag`. To test the failure path, uncomment `NvM_Block.maxSpeedLimit ^= 0xFFFFU;` before the second CRC calculation.